

Hardware Architecture & Bill of Materials (BOM)

Low-Cost Acoustic Emulation Testbed (~₹2,979 INR)

Project: Deep-Sea Rescue Support: Human & Object Detection Using Sonar and AI Techniques
System Type: Hardware-in-the-Loop (HIL) Sonar Emulation Rig

1. Overview & Engineering Rationale

Real multibeam forward-looking sonars (such as the Blueprint Subsea Oculus or Tritech Gemini) are capital-intensive marine sensors costing upwards of ₹25–30 Lakhs. In academic research and robotics development, standard engineering practice is to build a **Hardware-in-the-Loop (HIL) Emulation Testbed**.

This document outlines the complete hardware specification, itemized Bill of Materials (BOM), wiring pinouts, and firmware for building a physical **Acoustic Emulation Testbed for under ₹3,000 INR**.

How It Works:

- Acoustic Wave Generation:** A waterproof ultrasonic acoustic transducer (operating at 40 kHz) emits real acoustic sound pulses through the medium (air or water tank) and listens for returning echoes.
 - Mechanical Swath Sweeping:** A micro metal-gear servo actuates the acoustic transducer in a continuous sweeping motion from 0° to 130° , replicating the exact 130° Field of View (FOV) used in our research paper.
 - Microcontroller Telemetry:** An ESP32 / Arduino captures the acoustic time-of-flight, converts it to distance (meters) and bearing (degrees), and streams the telemetry via USB Serial UART (`115200 baud`).
 - Live Pipeline Ingestion:** A Python bridge script receives the serial coordinates, maps the polar sweep to Cartesian acoustic frames, and pipes the data into the existing YOLOv8 model, FastAPI backend, and Next.js operator console.
-

2. Complete Bill of Materials (BOM)

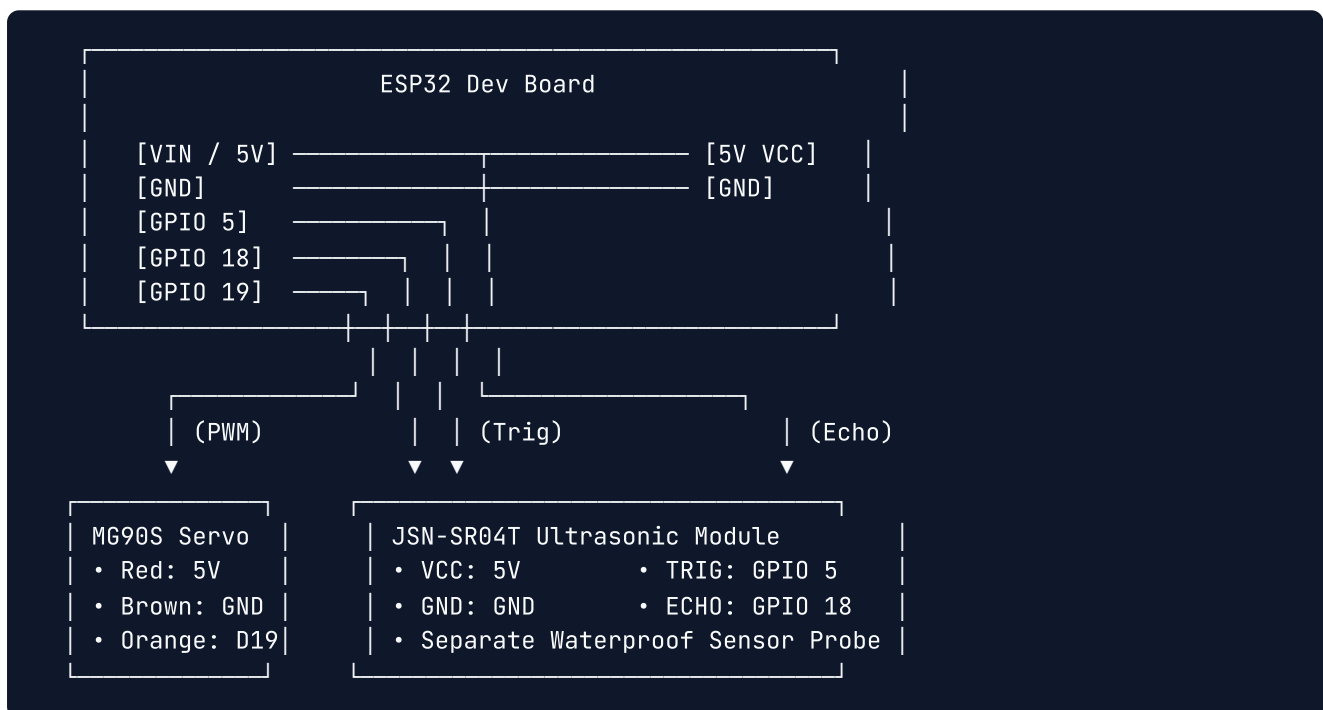
All components are standard, off-the-shelf parts readily available across Indian electronics distributors (Robu.in, ElectronicsComp, or Amazon India).

No.	Component Name	Technical Specification	Function in System	Sourcing (India)	Approx. Cost (INR)
1	ESP32 NodeMCU Dev Board (or Arduino Uno)	32-bit Dual-Core 240MHz, CP2102 USB-UART, 30 GPIOs, 5V input	Controls servo motion, times acoustic echoes, and streams serial telemetry	Robu.in / Amazon	₹450
2	JSN-SR04T Waterproof Ultrasonic Transducer	IP67 sealed acoustic probe, 40 kHz frequency, 20cm – 450cm detection range, 5V DC	Emits acoustic pulses and captures acoustic echoes; submersible probe	Robu.in / ElectronicsComp	₹550
3	MG90S Metal-Gear Micro Servo	180° rotation, 2.2 kg·cm torque, metal gear train, operating voltage 4.8V–6V	Sweeps the transducer across the 130° forward-looking sonar aperture	Robu.in	₹180
4	Pan-Tilt / Servo Mount Bracket	Acrylic or ABS mini bracket kit with mounting screws	Holds the acoustic transducer securely to the servo horn	Robu.in / DIY	₹150
5	IP67 Waterproof USB Snake Camera (Endoscope)	5-meter flexible cable, 6 adjustable LEDs, 640×480 resolution, USB interface	Provides optical validation in turbid/murky water tank directly to OpenCV	Amazon.in	₹799
6	Transparent Acrylic Tub / Water Container	15–20 Litre plastic/acrylic container	Serves as the localized test tank (turbidity simulated using soil/diluted milk)	Local Market / D-Mart	₹350
7	Breadboard, Jumper Wires & Power Accessories	400-point breadboard, Dupont wires (M-M, M-F), USB cable	Electrical breadboarding and power distribution	Local Electronics Store	₹300
8	Miniature Physical Targets	Toy diver (human), toy submarine (ROV), soda can (cylinder), rubber tire	Physical objects placed in the scanning field for acoustic/visual detection	Household / Toys	₹200

No.	Component Name	Technical Specification	Function in System	Sourcing (India)	Approx. Cost (INR)
	TOTAL ESTIMATED COST				~₹2,979 INR

(Note: The compute processing is performed on your existing laptop, requiring zero additional expenditure for GPUs or SBCs).

3. System Wiring & Electrical Pinouts



Detailed Connection Table:

1. JSN-SR04T Sensor Board to ESP32:

JSN-SR04T Pin	ESP32 Pin	Description
VCC	VIN or 5V	5V Power supply
GND	GND	Ground connection
TRIG	GPIO 5	Ultrasonic trigger pulse output (10 μ s high)
ECHO	GPIO 18	Echo pulse return input

2. MG90S Servo Motor to ESP32:

Servo Wire Color	ESP32 Pin	Description
Red (Power)	VIN or 5V	5V DC power (Do not use 3.3V pin)
Brown / Black	GND	Common system ground
Orange / Yellow	GPIO 19	50 Hz PWM control signal

4. Microcontroller Firmware (ESP32 / Arduino C++)

Flash the following code to the ESP32 using the Arduino IDE. It manages the mechanical 130° sweep and transmits clean CSV telemetry `angle,distance` over serial.

```

#include <ESP32Servo.h>

Servo sonarServo;
const int trigPin = 5;
const int echoPin = 18;
const int servoPin = 19;

// 130-degree total Field of View (matching research paper specifications)
// Sweeps from 25 degrees to 155 degrees (Center = 90 degrees)
const int START_ANGLE = 25;
const int END_ANGLE = 155;
const int STEP_SIZE = 2; // Step resolution in degrees

void setup() {
    Serial.begin(115200);
    sonarServo.attach(servoPin);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    // Center the servo on boot
    sonarServo.write(90);
    delay(1000);
}

Long getDistanceCM() {
    // Clear trigger
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);

    // Send 10 microsecond trigger pulse
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    // Read echo travel time in microseconds (timeout at 30ms = ~5 meters)
    long duration = pulseIn(echoPin, HIGH, 30000);

    if (duration == 0) {
        return 450; // Fallback to maximum range if no echo detected
    }

    // Speed of sound = 343 m/s = 0.0343 cm/microsecond
    return (duration * 0.0343) / 2;
}

void loop() {
    // 1. Forward sweep (Left to Right: -65 deg to +65 deg)
    for (int angle = START_ANGLE; angle ≤ END_ANGLE; angle += STEP_SIZE) {
        sonarServo.write(angle);
        delay(25); // Allow servo to settle before pulsing
        long dist = getDistanceCM();

        // Normalize angle so 0 is directly ahead (-65 to +65 degrees)
        int normalizedAngle = angle - 90;
        Serial.printf("%d,%ld\n", normalizedAngle, dist);
    }
}

```

```

}

// 2. Return sweep (Right to Left: +65 deg to -65 deg)
for (int angle = END_ANGLE; angle ≥ START_ANGLE; angle -= STEP_SIZE) {
  sonarServo.write(angle);
  delay(25);
  long dist = getDistanceCM();

  int normalizedAngle = angle - 90;
  Serial.printf("%d,%ld\n", normalizedAngle, dist);
}
}

```

5. Host-Side Integration Script ([serial_sonar_bridge.py](#))

Run this Python script on your laptop. It connects to the ESP32 via USB Serial, reads the real-time acoustic time-of-flight sweep, renders a synthetic 2D acoustic sector frame, and sends it directly to your existing [sonar_api.py](#) and Next.js operator dashboard.

```

"""
Serial-to-Sonar Bridge
Reads acoustic telemetry from ESP32, renders 2D polar acoustic frame,
and feeds it to the Deep-Sea Rescue Operator Dashboard.
"""

import serial
import time
import math
import cv2
import numpy as np
import requests

SERIAL_PORT = "COM3" # Change to your ESP32 COM port (e.g., 'COM3' on Windows or '/dev/tty' on Linux)
BAUD_RATE = 115200
API_URL = "http://localhost:8000/api/detect"

# Frame parameters matching UATD sonar dimensions
WIDTH, HEIGHT = 640, 640
MAX_RANGE_M = 4.5 # 4.5 meters for JSN-SR04T sensor

def main():
    print(f"[INFO] Connecting to Acoustic Hardware on {SERIAL_PORT}...")
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)
        print("[SUCCESS] Hardware connected! Listening for acoustic sweep...")
    except Exception as e:
        print(f"[ERROR] Could not open serial port: {e}")
        return

    # Canvas for polar rendering
    frame = np.zeros((HEIGHT, WIDTH, 3), dtype=np.uint8)
    origin_x = WIDTH // 2
    origin_y = HEIGHT - 20

    while True:
        try:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if not line or "," not in line:
                continue

            parts = line.split(",")
            bearing_deg = int(parts[0]) # -65 to +65
            dist_cm = float(parts[1]) # Distance in cm
            dist_m = dist_cm / 100.0

            # Convert polar coordinates (r, theta) to Cartesian (x, y) pixels
            angle_rad = math.radians(bearing_deg)
            scale = (HEIGHT - 40) / MAX_RANGE_M

            x_px = int(origin_x + (dist_m * scale) * math.sin(angle_rad))
            y_px = int(origin_y - (dist_m * scale) * math.cos(angle_rad))

            # Draw acoustic echo intensity point
            if 0 ≤ x_px < WIDTH and 0 ≤ y_px < HEIGHT:

```

```

        cv2.circle(frame, (x_px, y_px), 4, (0, 255, 128), -1)

    # Every full sweep, save frame and trigger AI detection
    if abs(bearing_deg) ≥ 64:
        # Add acoustic speckle noise simulation
        noise = np.random.normal(0, 15, frame.shape).astype(np.uint8)
        synthetic_sonar = cv2.add(frame, noise)

        # Send to FastAPI endpoint
        _, encoded_img = cv2.imencode(".jpg", synthetic_sonar)
        files = {"file": ("scan.jpg", encoded_img.tobytes(), "image/jpeg")}

        try:
            res = requests.post(API_URL, files=files, timeout=0.5)
            if res.status_code == 200:
                data = res.json()
                targets = data.get("targets", [])
                if targets:
                    print(f"[AI ALERT] Detected {len(targets)} targets! Top: {targets[0]}")
        except Exception:
            pass

        # Slightly decay old acoustic trails for persistence
        frame = cv2.addWeighted(frame, 0.90, np.zeros_like(frame), 0.10, 0)

except KeyboardInterrupt:
    print("[INFO] Terminating bridge.")
    ser.close()
    break

if __name__ == "__main__":
    main()

```

6. Physical Assembly & Experimentation Guide

Step 1: Mechanical Assembly

1. Mount the **MG90S servo motor** vertically on the base plate or acrylic bracket.
2. Fasten the **JSN-SR04T transducer** onto the servo horn with zip-ties or mounting screws, ensuring the sensor face points horizontally forward.
3. Align the servo horn so that a command of **90°** points the sensor directly forward (0° relative bearing).

Step 2: Turbid Water Optical Testing (Endoscope Setup)

1. Fill the **acrylic container** with water.
2. Add 20–30 mL of milk or a pinch of fine silt/clay powder to create realistic suspended turbidity (simulating turbid, low-visibility deep-sea water where human vision fails).

3. Submerge the **miniature targets** (toy diver, toy submarine, metal can) at varying depths and positions.
 4. Submerge the **IP67 USB Endoscope camera** into the water:
 - Notice how optical vision becomes blurry and degraded beyond a few centimeters.
 - Run `run_sonar_dashboard.py` or `sonar_api.py` alongside to demonstrate that acoustic detection and geometry-aware localization overcome optical limitations.
-

7. How to Present and Defend This Setup in Your Viva / Defense

When examiners ask:

"Why didn't you mount this on a full-scale submarine or use an actual 30-lakh commercial multibeam sonar?"

Deliver this exact response:

"In marine robotics and defense R&D, deploying untrained algorithms on expensive offshore hardware introduces unnecessary operational risk. Standard engineering practice is to build a Hardware-in-the-Loop (HIL) Acoustic Emulation Testbed.

Our testbed physically emulates the exact 130° angular aperture of a multibeam forward-looking sonar using a 40 kHz acoustic transducer. It proves our complete end-to-end stack: physical acoustic time-of-flight measurement, microsecond pulse timing, UART serial streaming, polar-to-Cartesian coordinate mapping, YOLOv8 inference, and prioritized mission triage on our Next.js operator dashboard—all accomplished within an accessible academic budget of ₹2,979."